Universidade da Beira Interior Departamento de Informática

Motor de Analytics para Dados de Retalho

Elaborado por:

Redinho Marques

Orientador:

VASCO FERRINHO LOPES

1 de maio de 2026

Conteúdo

1 Introdução **3**

2 Arquitetura do Sistema **3**

3 Estruturas de Dados Utilizadas **4**
3.1 Lista de Eventos 4
3.2 Tabela Hash 4
3.3 MaxHeap 4
3.4 Listas Auxiliares 5

4 Algoritmos Implementados **5**
4.1 Bubble Sort 5
4.2 Merge Sort 5
4.3 Pesquisa Binária 5
4.4 KMP 6

5 Queries Implementadas **6**

6 Funcionamento de Algumas Queries **7**
6.1 Picos de Ocupação 7
6.2 Top-K Zonas Mais Visitadas 7
6.3 Fluxo entre Zonas 7
6.4 Pesquisa de Sequência 7

7 Análise de Complexidade **8**

8 Benchmarks Empíricos **9**
8.1 Tempo de Execução das Queries 9
8.2 Comparação entre Bubble Sort e Merge Sort 9
8.3 Análise da Tabela Hash 9

9 Discussão **10**

10 Conclusão **10**

1 Introdução

Este trabalho consistiu no desenvolvimento de um motor de analytics para dados de retalho, baseado em eventos gerados por um sistema de visão computacional. O dataset representa a atividade de uma loja ao longo de vários dias, registando eventos do tipo `entry`, `exit` e `linger`. Cada evento contém informação temporal, zona da loja, género estimado e faixa etária estimada.

O principal objetivo foi construir um programa capaz de responder eficientemente a queries analíticas sobre um conjunto de dados de grande dimensão. Estas queries simulam funcionalidades típicas de um dashboard de retalho, permitindo analisar ocupação, permanência, picos de afluência, padrões de movimento e distribuição demográfica.

A abordagem adotada focou-se na implementação de estruturas de dados e algoritmos de raiz, respeitando os requisitos do trabalho. Foram implementadas estruturas como uma Tabela Hash e uma MaxHeap, bem como algoritmos de ordenação, pesquisa binária e pesquisa de padrões em strings.

A eficiência foi uma preocupação central, uma vez que o sistema deve conseguir responder a várias queries sem tempos de espera excessivos. Por isso, sempre que possível, foram usadas estruturas adequadas ao tipo de operação realizada.

2 Arquitetura do Sistema

O sistema foi organizado de forma modular, separando a leitura dos dados, armazenamento, processamento e execução das queries. Esta organização torna o código mais simples de compreender, testar e modificar.

O fluxo geral do sistema é o seguinte:

```
CSV
 ↓
Objetos Event
 ↓
Lista ordenada (Merge Sort)
 ↓
+----------------------+
| Hash | Heap | Listas |
+----------------------+
 ↓
Execução de Queries
 ↓
Output + Tempo de execução
```

Inicialmente, o ficheiro `events.csv` é lido e cada linha é convertida num objeto da classe `Event`. Esta classe guarda os atributos fundamentais de cada evento: identificador, timestamp, zona, tipo de evento, duração, género e faixa etária.

Após a leitura, os eventos são guardados numa lista e ordenados por timestamp através de Merge Sort implementado de raiz. Esta decisão é importante porque permite aplicar pesquisa

binária em queries que dependem de intervalos temporais. Assim, em vez de percorrer sempre todo o dataset, o programa consegue localizar rapidamente os limites de um intervalo.

O menu interativo permite ao utilizador selecionar cada query, introduzir parâmetros e visualizar os resultados formatados. Para cada query, o programa mede também o tempo de execução, permitindo avaliar empiricamente o desempenho.

3 Estruturas de Dados Utilizadas

3.1 Lista de Eventos

A lista de eventos é a estrutura base do programa. Todos os eventos lidos do CSV são armazenados nesta lista. Após a leitura, a lista é ordenada temporalmente por timestamp.

Esta estrutura foi escolhida porque permite armazenar todos os eventos de forma simples e percorrê-los sequencialmente quando necessário. Muitas queries exigem analisar todos os eventos ou um subconjunto deles, pelo que a lista é adequada para este tipo de processamento.

A principal vantagem da lista ordenada é permitir a utilização de pesquisa binária. Esta pesquisa é usada para localizar rapidamente o início e o fim de um intervalo temporal, reduzindo o custo de queries que filtram dados por data e hora.

3.2 Tabela Hash

Foi implementada uma Tabela Hash de raiz através da classe `TabelaHash`. Esta estrutura é usada para contagens frequentes, como número de entradas por zona, distribuição por género, distribuição por faixa etária e contagens por blocos horários.

A Tabela Hash foi escolhida porque permite inserção, atualização e consulta em tempo médio $O(1)$. Isto é muito útil em queries de agregação, onde é necessário contar repetidamente ocorrências associadas a uma chave.

Sem uma Tabela Hash, seria necessário procurar manualmente numa lista se uma determinada zona ou categoria já existia, o que aumentaria o custo computacional. Com hash, a operação de incremento torna-se mais eficiente.

A implementação também permite medir o *load factor* e o número de colisões. O *load factor* indica o nível de ocupação da tabela, enquanto as colisões indicam quantas chaves diferentes foram colocadas na mesma posição. Estas métricas são úteis para analisar a qualidade da função de hashing.

3.3 MaxHeap

Foi implementada uma MaxHeap de raiz para resolver queries de Top-K. Esta estrutura permite obter rapidamente os maiores elementos sem ordenar completamente todos os resultados.

A MaxHeap foi usada em queries como:

- Top-K zonas mais visitadas;
- Top-K períodos de maior afluência;

- Picos de ocupação;
- Zonas com maior tempo médio de permanência.

A inserção e remoção do maior elemento têm complexidade $O(\log n)$. Esta estrutura é adequada porque muitas queries apenas precisam dos maiores valores, não de uma ordenação completa.

3.4 Listas Auxiliares

Além das estruturas principais, foram usadas listas auxiliares para armazenar resultados temporários, distribuições por hora, sequências de zonas e pares chave-valor.

Estas listas são simples e úteis para operações intermédias, especialmente quando o número de elementos é reduzido ou quando é necessário construir dados antes de os inserir numa MaxHeap ou numa Tabela Hash.

4 Algoritmos Implementados

4.1 Bubble Sort

O Bubble Sort foi implementado como um dos algoritmos de ordenação exigidos. Este algoritmo compara pares adjacentes e troca-os quando estão fora de ordem.

A sua complexidade é $O(n^2)$, o que o torna pouco eficiente para grandes volumes de dados. No entanto, foi útil para comparação empírica com um algoritmo mais eficiente.

4.2 Merge Sort

O Merge Sort também foi implementado de raiz. Este algoritmo divide a lista em duas partes, ordena recursivamente cada metade e depois junta os resultados ordenados.

A sua complexidade é $O(n \log n)$, sendo mais adequado para datasets maiores. No projeto, o Merge Sort foi usado para ordenar contagens e eventos temporais.

A comparação com o Bubble Sort demonstrou a diferença prática entre os dois algoritmos, confirmando que o Merge Sort é mais eficiente.

4.3 Pesquisa Binária

A pesquisa binária foi implementada para encontrar o primeiro evento maior ou igual a uma data inicial e o último evento menor ou igual a uma data final.

Esta abordagem só é possível porque a lista de eventos está ordenada por timestamp. A complexidade da pesquisa binária é $O(\log n)$, sendo muito mais eficiente do que percorrer toda a lista.

Esta técnica foi usada em várias queries temporais, como ocupação por intervalo, comparação entre dias, Top-K por período e query composta.

4.4 KMP

O algoritmo KMP foi implementado para a pesquisa de sequências de zonas. Como o dataset não possui identificador único de pessoa, as sequências de visitas foram reconstruídas por perfil demográfico, usando a combinação género e faixa etária como proxy. Para cada perfil, é construída uma sequência de zonas com base nos eventos `entry`, sendo aplicado o algoritmo KMP para detetar padrões.

A complexidade do KMP é $O(n + m)$, onde n é o tamanho do texto e m é o tamanho do padrão. Isto evita recomparações desnecessárias e torna a pesquisa mais eficiente do que uma abordagem ingênua.

5 Queries Implementadas

O sistema implementa todas as queries obrigatórias:

1. Ocupação por zona num intervalo;
2. Média de permanência por zona;
3. Picos de ocupação;
4. Comparação entre dias;
5. Top-K zonas mais visitadas;
6. Top-K períodos de maior afluência;
7. Zonas com maior tempo médio de permanência;
8. Fluxo entre zonas;
9. Pesquisa de sequências de zonas;
10. Deteção de anomalias;
11. Query composta;
12. Perfil demográfico por zona.

Além destas queries, foram adicionadas duas opções auxiliares:

- comparação entre Bubble Sort e Merge Sort;
- estatísticas da Tabela Hash.

Estas opções adicionais foram úteis para cumprir a parte experimental do trabalho, permitindo medir tempos, comparações, load factor e colisões.

6 Funcionamento de Algumas Queries

6.1 Picos de Ocupação

A query de picos de ocupação usa uma abordagem semelhante a *sweep line*. Para cada evento do dia escolhido, o programa transforma:

- `entry` em $+1$;
- `exit` em -1.

Depois, os eventos são ordenados temporalmente através de Merge Sort. O programa percorre a lista e mantém uma ocupação acumulada. Cada valor acumulado representa o número de pessoas presentes naquele instante.

Os maiores valores são inseridos numa MaxHeap, permitindo extrair os K momentos de maior ocupação.

6.2 Top-K Zonas Mais Visitadas

Nesta query, o programa filtra os eventos por período, conta as entradas por zona usando a Tabela Hash e depois insere os resultados numa MaxHeap.

A escolha da Tabela Hash permite contar entradas de forma eficiente. A MaxHeap permite obter rapidamente as K zonas mais visitadas sem ordenar completamente todas as zonas.

6.3 Fluxo entre Zonas

Na query de fluxo entre zonas, o objetivo é identificar transições frequentes entre pares de zonas. Para isso, o programa procura eventos de `exit` seguidos por eventos de `entry` dentro de um threshold temporal definido pelo utilizador.

Como o dataset não possui identificador único de pessoa, a correspondência entre eventos de saída e entrada é feita usando género e faixa etária como proxy de identificação. Apenas são consideradas transições em que o evento de entrada ocorre dentro do limite temporal configurado.

6.4 Pesquisa de Sequência

Como o dataset não possui identificador único de pessoa, as sequências de visitas foram reconstruídas por perfil demográfico, usando a combinação género e faixa etária como proxy. Para cada perfil, é construída uma sequência de zonas com base nos eventos `entry`, sendo aplicado o algoritmo KMP para detetar padrões.

Esta abordagem demonstra a aplicação de algoritmos de pesquisa de padrões em strings a um problema real de análise de comportamento, mantendo coerência com a limitação do dataset e com a estratégia usada na query de fluxo entre zonas.

7 Análise de Complexidade

Query	**Complexidade**	**Justificação**
Ocupação por zona	$O(\log n + k + z \log z)$	Usa pesquisa binária para localizar o intervalo, percorre os eventos filtrados e ordena as zonas por contagem.
Média de permanência	$O(n)$	Percorre eventos `linger` e agrega os valores por hora.
Picos de ocupação	$O(m \log m)$	Considerando m eventos do dia, converte eventos em variações, ordena temporalmente e usa MaxHeap para obter os maiores picos.
Comparação entre dias	$O(\log n + k)$	Usa pesquisa binária para filtrar eventos dos dias e calcula totais, médias e zona mais visitada.
Top-K zonas	$O(\log n + k + z \log z)$	Usa pesquisa binária, Tabela Hash para contagem e MaxHeap para extrair os maiores valores.
Top-K períodos	$O(\log n + k + b \log b)$	Divide o período em blocos de 30 minutos, conta eventos e usa MaxHeap.
Maior permanência	$O(n + z \log z)$	Agrega duração por zona e usa MaxHeap para selecionar as maiores médias.
Fluxo entre zonas	$O(n^2)$ no pior caso	Procura entradas após saídas dentro de um threshold temporal. A correspondência entre eventos de saída e entrada é feita usando género e faixa etária como proxy de identificação.
Pesquisa de sequência	$O(n + m)$	Usa KMP para procurar padrões em sequências reconstruídas por perfil demográfico.
Anomalias	$O(n)$	Agrega entradas por zona e hora, calcula média e desvio padrão.
Query composta	$O(\log n + k)$	Filtra intervalo temporal e aplica filtros adicionais.
Perfil demográfico	$O(n)$	Conta entradas por género e faixa etária, podendo filtrar por zona, data específica e intervalo horário.

A análise mostra que a maioria das queries tem complexidade linear ou quase linear no número de eventos filtrados. A exceção principal é a query de fluxo entre zonas, que pode atingir $O(n^2)$ no pior caso.

8 Benchmarks Empíricos

Os tempos foram medidos diretamente no programa, usando a biblioteca `time`. Cada query imprime o seu tempo de execução após terminar.

8.1 Tempo de Execução das Queries

Query	**Tempo observado**
Ocupação por zona	0.03 s
Média de permanência	0.04 s
Picos de ocupação	0.238 s
Comparação entre dias	0.04 s
Top-K zonas	0.03 s
Top-K períodos	0.0097 s
Maior permanência	0.05 s
Fluxo entre zonas	1.00 s
Pesquisa de sequência	0.04 s
Deteção de anomalias	0.20 s
Query composta	0.03 s
Perfil demográfico	0.03 s

A query de fluxo entre zonas apresenta o maior tempo de execução, o que era esperado devido à sua complexidade mais elevada. Já as queries baseadas em contagens simples ou filtragem temporal apresentam tempos bastante reduzidos.

8.2 Comparação entre Bubble Sort e Merge Sort

Algoritmo	**Tempo**	**Comparações**
Bubble Sort	0.04783 s	276
Merge Sort	0.02644 s	67

Os resultados mostram que o Merge Sort foi mais eficiente do que o Bubble Sort, tanto no tempo de execução como no número de comparações. Isto confirma a diferença teórica entre os dois algoritmos: o Bubble Sort apresenta complexidade $O(n^2)$, enquanto o Merge Sort apresenta complexidade $O(n \log n)$.

Mesmo quando o número de zonas é relativamente pequeno, a diferença já é visível. Em datasets maiores, esta diferença tornar-se-ia ainda mais significativa.

8.3 Análise da Tabela Hash

Métrica	**Valor**
Tamanho da tabela	101
Load factor	0.1782
Colisões	5

O load factor obtido é baixo, indicando que a tabela não está demasiado preenchida. Isto reduz a probabilidade de colisões e melhora a eficiência das operações.

O número de colisões também é reduzido, o que sugere que a função hash distribuiu razoavelmente bem as chaves pelas posições da tabela. Assim, a Tabela Hash revelou-se adequada para as operações de contagem usadas no projeto.

9 Discussão

A escolha das estruturas de dados teve impacto direto no desempenho do sistema. A Tabela Hash foi especialmente importante em queries de agregação, pois permitiu contar elementos sem procurar repetidamente em listas.

A MaxHeap revelou-se adequada para queries Top-K, porque permite extrair os maiores elementos de forma eficiente. Sem heap, seria necessário ordenar todos os resultados, o que poderia ser mais dispendioso.

A pesquisa binária também teve papel importante nas queries temporais. Como os eventos estão ordenados por timestamp, foi possível localizar rapidamente intervalos de interesse e evitar varrimentos desnecessários sobre todo o dataset.

A comparação entre Bubble Sort e Merge Sort confirmou a importância da escolha correta do algoritmo de ordenação. O Bubble Sort é simples, mas pouco escalável. O Merge Sort, por outro lado, apresenta desempenho superior e é mais adequado para volumes de dados maiores.

Uma limitação do sistema é a query de fluxo entre zonas, que pode ser pesada em datasets grandes. Isto acontece porque é necessário procurar eventos de entrada após eventos de saída, dentro de um threshold temporal. Como não existe identificador único de pessoa, foi necessário usar género e faixa etária como proxy, o que pode introduzir alguma imprecisão.

Outra limitação é o facto de o sistema carregar todos os eventos em memória. Para datasets muito maiores, poderia ser necessário usar processamento por blocos ou armazenamento externo.

Como melhoria futura, poderiam ser implementadas estruturas de indexação adicionais, como índices por zona, por dia e por tipo de evento. Também seria possível otimizar a reconstrução de sequências de visitas e a deteção de fluxos entre zonas.

10 Conclusão

O sistema desenvolvido cumpre os objetivos do trabalho, permitindo ler o dataset, armazenar os eventos, executar as queries obrigatórias e apresentar os resultados formatados com o respetivo tempo de execução.

Foram implementadas de raiz estruturas de dados como Tabela Hash e MaxHeap, além de algoritmos como Bubble Sort, Merge Sort, pesquisa binária e KMP.

Os resultados empíricos demonstram que a escolha adequada das estruturas e algoritmos influencia diretamente o desempenho. Em particular, o uso de Tabela Hash, MaxHeap e pesquisa binária permitiu melhorar a eficiência de várias queries.

Conclui-se que o motor de analytics desenvolvido é funcional, organizado e adequado ao problema proposto, podendo servir como base para um sistema mais avançado de análise de

dados de retalho.